

Product Requirements Document

Arcane Event Ticketing Platform

Prepared by	Fern
Client	Arcane
Document version	2.0 (Research-deepened draft)
Date	13 June 2026
Status	For review
Market	Sri Lanka (primary)

1. Overview

Fern is building a web-based event ticketing platform for Arcane. Arcane publishes events; the general public discovers them and buys tickets online. Every purchased ticket generates a unique QR code at the moment of purchase. The QR is scanned at the venue on the event date to admit the holder, and a confirmation email containing the ticket is sent automatically with every successful purchase.

The **ticketing system is the priority for the first release** and is specified here in depth. Discovery, payments, QR generation, email delivery, and at-the-gate scanning form the core of Phase 1. This version incorporates research into QR-ticket security, the Sri Lankan payment-gateway landscape, high-demand on-sale architecture, email deliverability, digital wallet passes, and data-protection compliance.

2. Goals and Objectives

Primary goals

- Let Arcane create, publish, and manage events without engineering involvement.
- Let buyers browse events and complete a purchase in a few clicks.
- Generate a unique, fraud-resistant QR ticket on the spot for every ticket sold.
- Deliver a confirmation email with the ticket(s) on every successful purchase.
- Validate tickets reliably at the gate, including under poor venue connectivity.
- Survive high-demand on-sales without crashing or overselling.

Success metrics

- $\geq 95\%$ of purchases produce a ticket email delivered within 60 seconds.
- Gate scan-to-result under ~ 1 second; check-in throughput in single-digit seconds per attendee, well below manual entry.
- Zero double-admittance from a single valid ticket.
- Zero overselling beyond configured capacity, even at peak concurrency.
- Checkout completion $\geq 70\%$ of initiated checkouts.
- Event setup by an Arcane admin in under 10 minutes.

Non-goals (Phase 1)

- Reserved seating / seat maps (general admission only at launch).
- Resale / secondary marketplace.
- Native mobile apps (responsive web only; wallet passes covered separately).
- Multi-organizer marketplace (single-tenant for Arcane).

3. Personas

General User (Buyer) — Discovers events, buys tickets, receives them by email/wallet, presents the QR at the gate. Wants speed, trust, and easy ticket retrieval.

Arcane Admin (Organizer) — Creates/manages events, ticket types, and pricing; monitors sales and inventory. Wants control and live visibility.

Gate Staff (Scanner) — On event day, scans QR codes to admit attendees. Wants a fast, unambiguous valid/invalid signal that keeps working if venue Wi-Fi drops.

Fern (Platform Operator) — Maintains the platform, payment/email integrations, and escalations. Wants reliability and observability.

4. Scope by Phase

Capability	Phase 1 (MVP)	Phase 2	Phase 3
Event creation & publishing	Yes		
Event discovery (browse, filter, search)	Yes		
Ticket types & pricing	Yes		
Checkout & payment (local gateway)	Yes		
QR ticket generation (signed, single-use)	Yes		
Confirmation email with ticket	Yes		
Gate scanning / validation (offline-tolerant)	Yes		
Buyer “My Tickets” retrieval	Yes		
Inventory holds & oversell protection	Yes		
Refunds / cancellations	Manual	Automated	
Installment payments (e.g., Koko BNPL)		Yes	
Virtual waiting room (high-demand)	Basic throttle	Full queue	
Dynamic/rotating QR codes		Yes	
Sales & attendance analytics	Basic	Yes	
Ticket transfer, waitlists, loyalty, resale			Yes

5. Core User Flows

5.1 Buyer purchase flow

1. Buyer lands on the homepage; sees featured and upcoming events.
2. Browses by category, filters by date/price, or searches.
3. Opens an event detail page (description, venue, date/time, ticket types, prices, availability state).
4. Selects ticket type(s) and quantity; tickets are placed on a short hold.
5. Enters details (name, email, phone); proceeds to checkout.
6. Pays through the integrated gateway (hosted checkout).
7. On confirmed payment (server-side callback), the system issues a unique ticket + signed QR per ticket.
8. A confirmation email with the ticket(s) is sent automatically.
9. Buyer can retrieve tickets anytime via “My Tickets.”

5.2 Organizer event flow

1. Admin logs into the Arcane admin panel.
2. Creates an event (title, description, banner, venue, date/time, category).
3. Defines ticket types (name, price, quantity, per-order limit, sale window).
4. Previews and publishes.
5. Monitors live sales, revenue, and remaining inventory.
6. Edits, pauses sales, or unpublishes as needed.

5.3 Gate scanning flow

1. Gate staff open the scanner web app (camera-enabled) and select the active event.
2. The device pre-loads the event's valid-ticket set for offline tolerance.
3. Staff scan the attendee's QR.
4. The system validates: Valid → admit and lock; Already used; Invalid/forged; or Wrong event.
5. A valid ticket is marked used immediately so it cannot be reused.
6. Admissions sync to the backend; multiple gates stay consistent.

6. Functional Requirements

6.1 Event Discovery (public site)

- **FR-1.1** Homepage shows featured/trending events and an upcoming-events grid.
- **FR-1.2** Browse by configurable category (e.g., Concerts, EDM, Family).
- **FR-1.3** Filter by date range and price; keyword search.
- **FR-1.4** Event cards show banner, date, start time, title, venue, “starting at” price, and a clear CTA.
- **FR-1.5** Event detail page shows full description, venue, date/time, ticket types with prices, and availability state (“Selling fast,” “Sold out”).
- **FR-1.6** Mobile-first responsive design — the majority of buyers will be on phones.

6.2 Ticketing, Inventory & Oversell Protection

- **FR-2.1** Each event supports one or more ticket types: name, price, total quantity, optional per-order limit.
- **FR-2.2** Inventory must never be oversold beyond configured capacity under any level of concurrency.
- **FR-2.3** Selected tickets are placed on a time-boxed hold (e.g., 10 minutes) during checkout; the hold auto-releases if checkout is not completed.
- **FR-2.4** Inventory decrements are atomic. Design against the known failure mode: a cache and database falling out of sync lets two buyers each see “1 remaining” and both purchase. Use atomic counters / an inventory-token approach (e.g., Redis atomic operations as source of truth for availability) with the database as the durable record.
- **FR-2.5** Order creation and payment handling are idempotent — a retried request or duplicate gateway callback never issues a second ticket or double-charges.
- **FR-2.6** Ticket types can have a sale start/end window and be hidden or disabled.
- **FR-2.7** Each issued ticket has a globally unique identifier linked to its event, type, order, and buyer.

6.3 QR Code Generation & Security

- **FR-3.1** On confirmed payment, generate one unique QR per ticket, on the spot.
- **FR-3.2** The QR encodes a non-guessable, tokenized identifier — never raw personal data (no name, phone, or seat in the payload). The token maps server-side to the ticket record.
- **FR-3.3** Each ticket token carries a cryptographic signature; validation verifies the signature and a server-side lookup, so a fabricated or altered code fails.
- **FR-3.4** Tickets are single-use for admission: the first valid scan locks the ticket (issued → valid → used / void). A second presentation is rejected.
- **FR-3.5** Anti-fraud coverage — resist common attacks on static tickets: screenshotting and sharing, reselling the same PDF to several buyers, a group passing one code and racing different gates, printed-copy reuse, and “forward the email to the group” harvesting. Phase 1 mitigates via unique signed single-use tokens plus real-time cross-gate sync (first valid scan wins, instantly locked everywhere). Phase 2 adds dynamic/rotating QR codes that refresh on a short interval (industry implementations rotate roughly every 15 seconds), so a screenshot expires almost immediately.
- **FR-3.6** Every QR carries a visible, branded call-to-action (“Scan to enter”) and Arcane branding, which improves scan trust and rates.

Why this matters: ticket fraud is a material revenue and reputation risk, and well-implemented QR validation both reduces fraud and dramatically speeds entry (industry data points to check-in times dropping from ~45s to single-digit seconds and error rates from ~3.5% to ~0.5% versus manual entry).

6.4 Email Delivery & Deliverability

- **FR-4.1** A confirmation email is sent automatically on every successful purchase.
- **FR-4.2** The email contains order details and the ticket(s) with QR code(s) — recommend both an embedded QR (convenience) and an attached PDF (offline/print).
- **FR-4.3** Send via a reputable transactional email provider with delivery tracking and automatic retry on transient failure.
- **FR-4.4** Authentication is mandatory: configure SPF, DKIM, and DMARC on a branded Arcane sending domain (not a shared ESP domain). Major inbox providers filter or block mail lacking these, and bulk senders are held to low spam-complaint thresholds (well under ~0.3%). Roll DMARC out safely (monitor with p=none, then tighten toward quarantine/reject).
- **FR-4.5** Monitor reputation via provider dashboards / Google Postmaster Tools, especially around large on-sales.
- **FR-4.6** Buyers can re-send / re-download their ticket from “My Tickets” if the email is lost — the email is a convenience channel, not the only path to the ticket.
- **FR-4.7** Email templates are branded for Arcane.

6.5 Payments (Sri Lankan market)

FR-5.1 Integrate at least one Sri Lankan internet payment gateway (IPG) supporting local Visa/Mastercard and common local wallets. Candidate gateways:

Gateway	Notes
PayHere	Most widely adopted by SL SMEs; cards plus local wallets (HelaPay, Frimi, Genie, iPay, Q Plus); settles to any local bank; supports foreign currency for diaspora; official SDK, sandbox, IPN callbacks.
OnePay	First SL gateway with ISO/IEC 27001 certification; developer-first (JS checkout, REST APIs, plugins, mobile SDKs); already powers a ticketing product (OneTicket).
WebXPay	Early SL IPG; works with or without a full website.
Bank IPGs (Sampath, Commercial, HNB, etc.)	Often better for reliable multi-currency and faster payouts at higher volumes.
Koko (BNPL)	“Pay in installments” — mirrors the installment option on comparable local platforms; Phase 2.

Working recommendation: launch on PayHere or OnePay for strongest local coverage and developer experience; add Koko installments in Phase 2.

- **FR-5.2** Tickets are issued only after server-confirmed payment success, via the gateway’s IPN/webhook callback — never on the client-side redirect alone.
- **FR-5.3** Failed or abandoned payments release any held inventory.
- **FR-5.4** Each order records an auditable transaction reference and gateway status.
- **FR-5.5** Card data never touches Fern’s servers — use the gateway’s hosted/PCI-compliant checkout.
- **FR-5.6** (Phase 2) Installment payments via Koko or equivalent.

6.6 Gate Scanning & Validation

- **FR-6.1** A camera-based scanner (web app) reads and validates QR codes in real time.
- **FR-6.2** Validation returns a clear, fast result: Valid (admit), Already Used, Invalid/Forged, Wrong Event.
- **FR-6.3** On a valid scan, the ticket is immediately locked to prevent reuse — “first valid scan wins.”
- **FR-6.4** Offline tolerance: the scanner caches the event’s valid-ticket set so it keeps validating during intermittent connectivity, then syncs admissions on reconnect. Conflict handling resolves a ticket scanned at two gates while offline (flag the second as a duplicate on sync).
- **FR-6.5** Multi-device consistency: several gate devices can operate for one event simultaneously without admitting the same ticket twice.
- **FR-6.6** Every scan is logged (ticket, time, device/gate, result) for post-event reconciliation.
- **FR-6.7** No exceptions to scanning — even VIP/staff/complimentary passes carry QR codes and are scanned.

6.7 Buyer Account / Ticket Retrieval

- **FR-7.1** Tickets retrievable under “My Tickets,” ideally without forced account creation (e.g., email magic-link lookup), with optional sign-up.
- **FR-7.2** Order history viewable; tickets re-downloadable.
- **FR-7.3** (Phase 2) Add to Apple Wallet / Google Wallet — passes can be updated remotely via push (e.g., venue/time change notifications) and surface by time/location. Implementable via the native Apple/Google Wallet APIs or a provider such as PassKit.

6.8 Admin Panel

- **FR-8.1** Role-based access for Arcane admins.
- **FR-8.2** CRUD for events and ticket types.
- **FR-8.3** Live view of sales, revenue, and remaining inventory per event.
- **FR-8.4** Manual refund/cancellation (Phase 1) that voids the associated ticket(s) and QR(s); automated refunds in Phase 2.
- **FR-8.5** Export of attendee/sales lists.
- **FR-8.6** Post-event reconciliation view (issued vs. scanned vs. no-shows vs. duplicates).

6.9 High-Demand On-Sale Handling

- **FR-9.1** Popular Arcane events can sell out in minutes, concentrating traffic into a short window. The platform must stay up and avoid overselling under this load.
- **FR-9.2** Virtual waiting room (Phase 2 full, Phase 1 basic throttle): when demand exceeds a threshold, route new users into a metered queue and admit them to checkout in controlled batches, giving the backend time to lock inventory. Show queue position/estimated wait.
- **FR-9.3** Architectural elasticity: cloud auto-scaling and load balancing; aggressive CDN/in-memory caching of event pages; asynchronous queuing of database writes so the transactional backend doesn't lock up at peak.
- **FR-9.4** Bot defense: deprioritize/block automated traffic at the edge so real buyers reach inventory.
- **FR-9.5** Load testing well above expected peak before any major on-sale.
- **FR-9.6** Graceful degradation: non-essential dynamic elements can be cached or temporarily disabled to keep the payment and inventory paths fully operational.

7. Non-Functional Requirements

- **Performance:** Event pages load fast on mobile networks; QR validation responds under ~1 second on a normal connection.
- **Scalability:** Handle on-sale spikes with consistent inventory under concurrency.
- **Reliability:** No overselling; no double-admittance; payment and email flows retried and idempotent; recovery strategies for cache/queue/service failures (fallback to DB, rehydrate cache, retry from queue, compensation logic to release held seats).
- **Security:** Non-guessable signed QR tokens; PCI-compliant hosted payment; encryption in transit and at rest; least-privilege admin roles.
- **Availability:** High uptime, hardened around on-sale and event-day windows; multi-region redundancy where feasible.
- **Auditability:** Every order, payment, ticket state change, and scan is logged.
- **Accessibility:** Public site meets common accessibility standards (contrast, keyboard navigation, alt text).

8. Compliance — Sri Lanka PDPA (No. 9 of 2022)

The platform processes personal data (buyer name, email, phone), so it falls under Sri Lanka's Personal Data Protection Act, enforced by the Data Protection Authority (established 2023).

Requirements to build in:

- **Lawful, transparent processing** with a clear privacy notice stating the controller, purpose of processing, who data is shared with (payment gateway, email provider), and any cross-border transfer.
- **Consent & purpose limitation** — collect only what's needed for ticketing; obtain consent for marketing use, with easy withdrawal.
- **Data subject rights** — provide a path to access/correct/erase data; the Act sets response timeframes (around 21 business days for certain requests).
- **Data minimization** — don't store more than necessary; keep the QR payload free of personal data.
- **Security & breach notification** — appropriate safeguards; notify the Authority and affected data subjects of a breach.
- **Cross-border transfers** — if providers store data outside Sri Lanka, ensure an adequacy basis or valid exception (consent, contract performance).
- **Registration** — confirm whether Arcane must register as a controller with the Authority.

Fern should confirm current PDPA operational requirements with Arcane's legal counsel before launch; specifics of enforcement and registration continue to evolve.

9. High-Level Architecture (proposed)

- **Public web app** — discovery, event detail, checkout, “My Tickets” (responsive, CDN-fronted).
- **Admin web app** — event/ticket management, sales monitoring, reconciliation.
- **Scanner web app** — camera-based validation, offline-tolerant cache + sync.
- **Backend API & services** — events, ticket types, inventory, orders, tickets, scans; enforces atomic inventory and validation rules.
- **Inventory/cache layer** — atomic counters / token inventory (e.g., Redis) as the fast source of truth for availability; durable DB as the record of sales.
- **Async queue** — decouples spike traffic from durable writes; enables retry and compensation.
- **Payment integration** — hosted checkout + server-side IPN/webhook confirmation.
- **Transactional email service** — authenticated (SPF/DKIM/DMARC) branded sending domain, retries, reputation monitoring.
- **QR service** — signed token issuance + QR rendering; (Phase 2) rotating-code endpoint.
- **Wallet pass service (Phase 2)** — Apple/Google Wallet pass issuance and push updates.
- **Object storage** — event images, generated ticket PDFs.
- **Virtual waiting room (Phase 2)** — edge-metered admission with bot filtering.

10. Core Data Entities (indicative)

- **Event** — id, title, description, banner, venue, datetime, category, status.
- **TicketType** — id, event_id, name, price, total_qty, sold_qty, held_qty, per_order_limit, sale_window.
- **Order** — id, buyer (name, email, phone), amount, gateway_ref, status, idempotency_key, created_at.
- **Ticket** — id, order_id, event_id, ticket_type_id, qr_token (signed), state (issued/valid/used/void), used_at, scanned_by.
- **ScanLog** — id, ticket_id, event_id, device/gate, result, timestamp, synced_at.
- **Hold** — id, ticket_type_id, qty, session_ref, expires_at.
- **AdminUser** — id, role, credentials.

11. Key Design Decisions to Confirm with Arcane

1. **Payment gateway(s)** — Assumption: launch on PayHere or OnePay; Koko installments in Phase 2. Which does Arcane prefer / already have a merchant account with?
2. **Currency** — Assumption: LKR primary. Any need for foreign-currency sales to diaspora buyers?
3. **Seating** — Assumption: general admission only at launch. Any reserved-seating events planned?
4. **Account model** — Assumption: guest checkout + email magic-link retrieval. Mandatory accounts wanted?
5. **Refund policy** — Assumption: manual refunds in Phase 1. What's the policy and who approves?
6. **Dynamic QR & wallet passes** — Assumption: Phase 2. Acceptable, or needed at launch?
7. **Expected scale** — peak concurrent buyers at on-sale and attendees per event, to size waiting-room and scanning needs.
8. **Branding & taxonomy** — Arcane to provide brand assets, email templates, and event categories.
9. **PDPA** — confirm controller registration status and privacy-notice content with legal counsel.

12. Risks & Mitigations

Risk	Impact	Mitigation
On-sale traffic spike	Crash, lost sales	Waiting room, auto-scaling, CDN caching, async writes, load testing
Cache/DB race	Overselling	Atomic inventory counters, idempotent orders, holds with TTL
Static QR sharing/screenshots	Double entry, fraud	Signed single-use tokens, cross-gate sync; Phase 2 rotating QR
Venue connectivity failure	Gate can't validate	Offline-tolerant scanner with cached ticket set + reconcile on sync
Payment confirmed but email fails	Buyer has no ticket	Webhook-driven issuance + retries + "My Tickets" fallback
Forged/duplicated codes	Unauthorized entry	Cryptographically signed, server-validated tokens
Email landing in spam	Tickets not received	SPF/DKIM/DMARC on branded domain, reputation monitoring
PDPA non-compliance	Legal/financial penalty	Privacy notice, consent, minimization, breach process, legal sign-off

13. Acceptance Criteria (Phase 1 highlights)

- An admin creates and publishes an event with multiple ticket types in under 10 minutes.
- A buyer completes a purchase end-to-end on mobile and receives a ticket email within 60 seconds.
- Each issued ticket carries a unique, signed, validatable QR with no personal data in the payload.
- Gate staff scan and admit a valid ticket; a second scan of the same ticket is rejected.
- The scanner keeps validating during a temporary connectivity loss and reconciles on reconnect; the same ticket cannot be admitted at two gates.
- No event is oversold beyond configured capacity under concurrent load (verified by load test).
- Tickets are issued only after server-confirmed payment, and never duplicated on retries/duplicate callbacks.
- Confirmation email passes SPF/DKIM/DMARC and lands in the inbox in deliverability testing.
- A buyer who lost the email retrieves the ticket via "My Tickets."